Project Proposal: Distributed IoT Intrusion Detection & Network Traffic Analytics using spark.

Problem Statement & Objectives

Modern smart-home and industrial Internet of Things (IoT) ecosystems generate massive streams of network packet logs. Single-threaded data analysis tools like Pandas fail to process these high-velocity datasets efficiently, leading to severe memory bottlenecks and slow detection of malicious cyberattacks. This project addresses the challenge of building an end-to-end distributed network security processing pipeline using PySpark on a standalone cluster in Google Colab (one master, multiple worker processes). Specifically, we solve three key tasks:

1. Standardize, clean, and engineer high-dimensional continuous and categorical network attributes across over 120,000 traffic logs.
2. Compute distributed group summary statistics and pairwise Pearson correlation matrices to identify key network anomaly indicators.
3. Train predictive ML models to classify malicious traffic vectors while quantifying speedup (T_1/T_n) across cluster executor scaling.

Dataset Description and Source

- Dataset: Real-Time IoT Network Traffic Dataset (RT_IOT2022)
- Details: 123,117 rows and 85 attributes capturing packet durations, flow rates, TCP flags, payload statistics, and target Attack_type categories (e.g., DoS, ARP Poisoning, NMAP Scans). Exceeds the 50,000-record threshold without synthetic resampling.
- **Kaggle Dataset:** RT-IoT2022 on Kaggle

Implementation Plan & Proposed PySpark Methods

- Beven Mpofu (Preprocessing): Clean data, encode categorical variables (StringIndexer, OneHotEncoder), and scale continuous features into dense vectors using VectorAssembler and StandardScaler (pyspark.sql, pyspark.ml.feature).
- Memory Mhembere (Statistical Analysis): Calculate group summary statistics using pyspark.sql.functions and compute Pearson correlation matrices across numeric features using Correlation.corr() (pyspark.ml.stat).
- Sibonginkosi T Nkashe (Machine Learning): Train a multi-class LogisticRegression model on an 80/20 train/test split and evaluate attack classification performance using MulticlassClassificationEvaluator (pyspark.ml.classification, pyspark.ml.evaluation).

Executor Scaling Experiment Plan

Each team member will execute their module under three PySpark cluster configurations: 1 Executor, 2 Executors, and 4 Executors (1 CPU core each). Each trial will run for ≥ 3 repeats to record wall-clock runtime mean and standard deviation ($\mu \pm \sigma$). Partition counts and individual speedup curves (T_1/T_n) will be documented and synthesized into a unified team table and figure comparing scaling performance across preprocessing, statistical analysis, and ML workloads.